

Task 02 — React API Project: Weather

Application

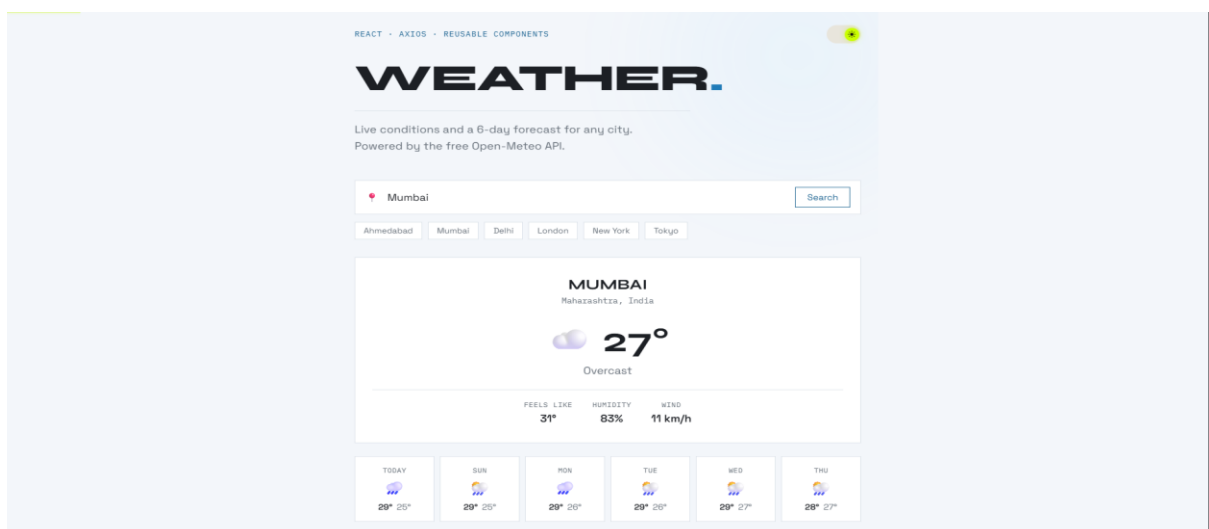
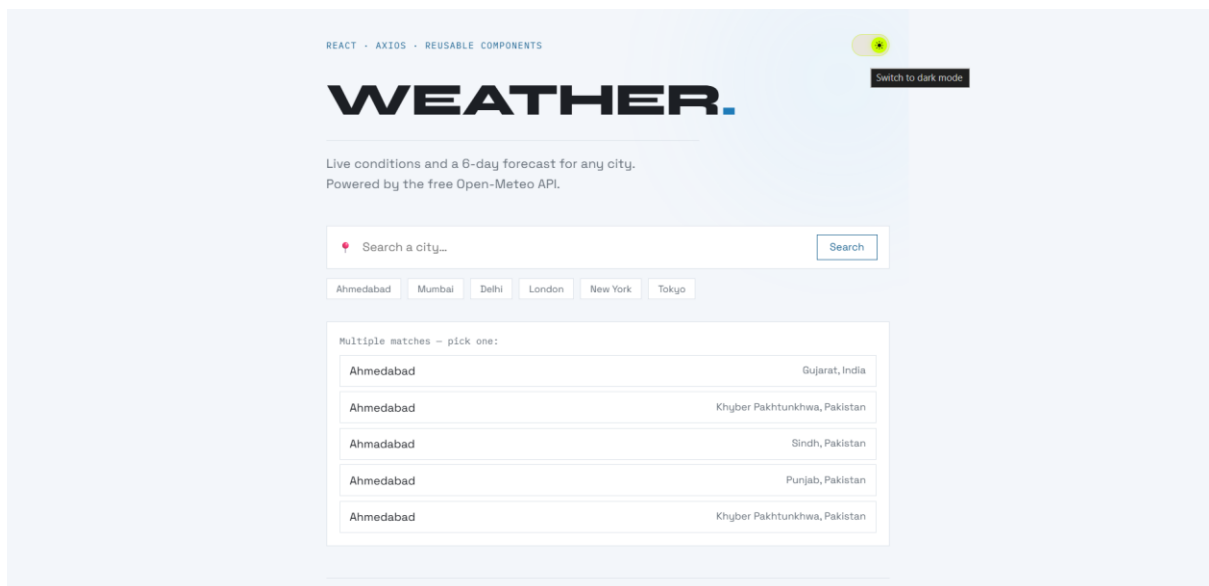
Objective

o develop a React-based Weather Application that fetches real-time weather information from an external API and displays the data using reusable React components. The application includes city search, loading and error states, current weather information, and a 6-day forecast.

Technologies Used

1. React 18
2. Vite
3. JavaScript
4. Axios
5. Open-Meteo API
6. CSS
7. GSAP
8. Vercel

Screenshots



API Integration Using Axios

- **Code Snippet — Axios API Service**

```
import axios from 'axios'
```

```
const geoClient =  
  axios.create({  
    baseURL:  
    'https://geocoding-  
api.open-meteo.com/v1',  
    timeout: 8000,  
  })
```

```
const weatherClient =  
  axios.create({  
    baseURL:  
    'https://api.open-  
meteo.com/v1',  
    timeout: 8000,  
  })
```

```
export async function  
searchLocations(name) {  
  const response = await  
    geoClient.get('/search', {  
      params: {  
        name,  
        count: 5,  
        language: 'en',  
        format: 'json',  
      },  
    })
```

```
    return  
    response.data.results ?? []  
  }
```

```
export async function  
getWeather(latitude,  
longitude) {  
  const response = await  
    weatherClient.get('/forecas  
t', {  
      params: {  
        latitude,  
        longitude,  
        current:
```

```
'temperature_2m,relative_  
humidity_2m,apparent_te  
mperature,weather_code,  
wind_speed_10m,is_day',  
      daily:
```

```
'weather_code,temperatur  
e_2m_max,temperature_2  
m_min',  
      timezone: 'auto',  
      forecast_days: 6,  
    },  
  })
```

```
    return response.data  
  }
```

Explanation:

Axios is used to communicate with the Open-Meteo API. Two Axios clients are configured: one for city geocoding and another for weather data. `searchLocations()` converts a city name into geographic coordinates, while `getWeather()` uses latitude and longitude to retrieve current weather conditions and a 6-day forecast. An 8-second timeout is also configured to prevent requests from hanging indefinitely.

Snippet 2 — React Data Fetching + Loading/Error States

```
async function runSearch(cityName) {          return
  setStatus('loading')                        }
  setErrorMsg('')
  setLocationOptions(null)                   if (results.length === 1) {
                                              await loadWeatherFor(results[0])
  try {                                       } else {
    const results = await                    setLocationOptions(results)
    searchLocations(cityName)                setStatus('idle')
                                              }
    if (results.length === 0) {              } catch (err) {
      setStatus('error')                     setStatus('error')
      setErrorMsg(                           setErrorMsg(describeError(err))
        `No results for "${cityName}". Try a
        different spelling.`
      )                                       }
    }                                       }
```

Explanation:

The `runSearch()` function manages the complete search flow. The application first changes the state to loading, performs the API request, and then handles different outcomes. If no city is found, an error message is displayed. If multiple locations are found, the user is given a choice. Network or server errors are caught using `try...catch` and displayed through the application's error state.

Snippet 3 — Reusable Weather Component

export default function	<div className="weather-
WeatherCard({ location,	card__main">
current }) {	<span
const { label, icon } =	className="weather-
	card__icon">
describeWeatherCode(curren	{icon}
t.weather_code)	
return (	<span
<article	className="weather-
className="weather-card">	card__temp">
<div className="weather-	
card__location">	{Math.round(current.tempera
	ture_2m)}°
{location.name}	
	</div>
{[location.admin1,	
location.country]	{label}
.filter(Boolean)	</article>
.join(', ')}>	)
	}
</div>	

Explanation:

WeatherCard is a reusable functional React component that receives weather information through props. It displays the selected location, current temperature, weather condition, and weather icon. Separating this functionality into its own component keeps the main App component organized and makes the weather display reusable.

Snippet 4 — 6. Search Component

```
export default function
SearchBar({ onSearch, loading
}) {
  const [value, setValue] =
    useState("")

  function handleSubmit(e) {
    e.preventDefault()

    const trimmed =
    value.trim()

    if (trimmed) {
      onSearch(trimmed)
    }
  }

  return (
    <div
      className="searchbar">
```

```
    <form
      onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Search a
        city..."
        value={value}
        onChange={(e) =>
          setValue(e.target.value)}
      />

      <button type="submit"
        disabled={loading}>
        {loading ? '...' : 'Search'}
      </button>
    </form>
  </div>
)
```

Explanation:

The SearchBar component manages the city search input using React's useState hook. When the form is submitted, the entered city name is passed to the parent component through the onSearch callback prop. The search button is disabled while the API request is loading, providing feedback to the user.

Snippet 5 — Search Component

```
export default function ForecastList({ daily          {icon}
}) {                                                </span>

  if (!daily?.time?.length) return null

                                                    <span>

  return (                                          {Math.round(
    <div className="forecast">                        daily.temperature_2m_max[i]
      {daily.time.map((date, i) => {                })°
        const { icon, label } =

                                                    {Math.round(
describeWeatherCode(daily.weather_code             daily.temperature_2m_min[i]
[i])                                                  })°

                                                    </span>

    return (                                       </div>
      <div key={date}                               </div>
        className="forecast__day">                  )
      <span>                                         <span>
        {formatDay(date, i)}                        {formatDay(date, i)}
      </span>                                       </div>
    )                                              )
  }                                              }

  <span title={label}>
```

Explanation:

The ForecastList component receives daily forecast data through props and dynamically creates a forecast item for each day using `.map()`. This makes the component reusable for different API responses and avoids manually creating individual forecast elements. Weather codes are converted into readable conditions using the application's weather-code utility.

Snippet 6 — API Documentation Notes

API Used

Open-Meteo API

The application uses two API endpoints.

Endpoint	Method	Purpose
/v1/search	GET	Converts a city name into location information
/v1/forecast	GET	Retrieves current weather and daily forecast

Geocoding API

<https://geocoding-api.open-meteo.com/v1/search>

Used to search for cities and obtain:

- City name
- Country
- Region
- Latitude
- Longitude

Weather API

<https://api.open-meteo.com/v1/forecast>

Used to retrieve:

- Current temperature
- Feels-like temperature
- Humidity
- Wind speed
- Weather code
- Daily maximum temperature
- Daily minimum temperature

Snippet 7 — Loading & Error UI

```
{status === 'loading' && (  
  <Loader label="Fetching weather..." />  
)}  
  
{status === 'error' && (  
  <ErrorMessage message={errorMsg}>  
    <button  
      onClick={() =>  
        runSearch(location?.name ?? DEFAULT_CITY)  
      }  
    >  
      Try again  
    </button>  
  </ErrorMessage>  
)}
```

Explanation:

The application provides dedicated UI states for API loading and failures. While data is being fetched, the Loader component is displayed. If the API request fails or the city cannot be found, the ErrorMessage component displays an appropriate error message and provides a retry option.

Live Demo:

<https://weatherrapi.vercel.app>

GitHub Repository

<https://github.com/Aditya00-git/task02>